The Hashemite University, Zarqa, Jordan
Faculty of Prince Al-Hussein Bin Abdullah II for Information Technology
Information Technology Department

Security Threats on the Electronic Bracelets for the Convicts in Jordan

A project submitted
in partial fulfillment of the requirements for the
B.Sc. Degree in Cyber Security

By

Saja Abdallah Alqawareeq (2232470)
Laila Ali Harb (2231818)
Rawan Mohammad Abed (2232539)
Maram Husam Zaza (2239667)
Hanan Hazim Alabsa (2232019)

Supervised by

Dr. Musab Alghadi

Committee Member Names
Prof. Ibrahim Obeidat

December 2025

CERTIFICATE

This is to certify that the project titled **" *Security Threats on the Electronic Bracelets for the Convicts in Jordan*"**, submitted by the undersigned, is a record of original work carried out by them under my supervision, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Cyber Security**.

All the analysis, design, implementation, and system development work has been carried out by the undersigned. Furthermore, this project has not been submitted, either in whole or in part, to any other college or university for the award of any degree or diploma.

Saja Abdallah Alqawareeq	***(2232470)***	*Signature*
Laila Ali Harb	***(2231818)***	*Signature*
Rawan Mohammad Abed	***(2232539)***	*Signature*
Maram Husam Zaza	***(2239667)***	*Signature*
Hanan Hazim Alabsa	***(2232019)***	*Signature*

ABSTRACT

Electronic bracelets are IoT-based devices widely used for monitoring and tracking individuals in applications such as healthcare, law enforcement, and personal safety. Due to their continuous connectivity and reliance on wireless communication, these devices are exposed to various cybersecurity threats, including unauthorized access, data leakage, and device manipulation. Ensuring the security of electronic bracelets is critical, as any compromise may lead to privacy violations or system misuse.

The objective of this project is to analyze the cybersecurity risks associated with electronic bracelet systems and to propose a secure framework that enhances data protection, device integrity, and communication security. The project aims to identify potential vulnerabilities and evaluate their impact on system reliability and user safety.
The methodology involves studying existing IoT security models, analyzing relevant datasets from similar IoT or autonomous systems, and applying threat modeling and vulnerability assessment techniques. Security mechanisms such as encryption, authentication, and secure communication protocols are considered in the proposed solution.

The results highlight common vulnerabilities in IoT-based monitoring devices and demonstrate how applying appropriate security controls can significantly reduce attack surfaces. This project contributes to improving the security design of electronic bracelets and emphasizes the importance of cybersecurity in IoT systems with real-world impact.

TABLE OF CONTENTS

ABBREVIATIONS

- **GPS**: Global Positioning System
- **EM:** Electronic Monitoring
- **INS**: Inertial Navigation System
- **AV**: Autonomous Vehicle
- **CSV**: Comma-Seperated Values
- **sklearn**: Scikit-learn
- **Agg:**Anti-Grain Geometry

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Overview

Electronic monitoring bracelets are widely used in judicial and security systems to track individuals using GPS-based location monitoring. However, these devices are vulnerable to GPS spoofing attacks, where forged satellite signals mislead the bracelet into reporting a false location. This allows an individual to appear within an authorized area while actually being outside it, posing serious risks to public safety and reducing the reliability of electronic monitoring systems. This project focuses on analyzing how GPS spoofing attacks are performed, identifying the technical weaknesses of electronic bracelets, and improving spoofing detection mechanisms to enhance the accuracy and security of GPS-based tracking systems.

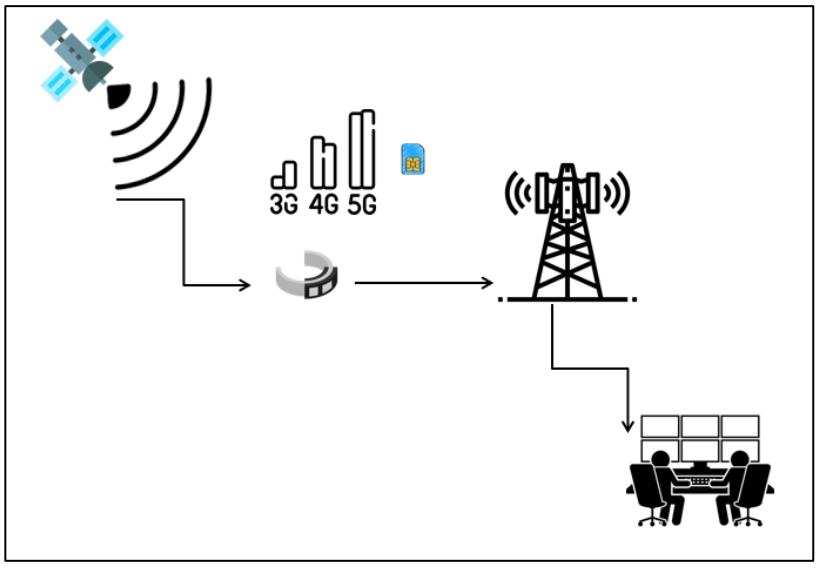

Figure 1. Electronic Bracelet Architecture.

The architecture illustrates the data flow and communication components of the electronic monitoring system:

1. GPS Satellite
   GPS satellites transmit positioning signals used by the electronic bracelet to calculate the wearer's location.
2. Electronic Monitoring Bracelet
   The bracelet receives GPS signals, determines the user's location, and prepares the data for transmission.
3. Cellular Network (3G / 4G / 5G with SIM Card)
   Location data is transmitted from the bracelet to the backend system through cellular networks using an embedded SIM card.
4. Monitoring and Control Center
   The control center receives and analyzes location data in real time, allowing authorities to verify compliance with movement restrictions.

GPS and Movement Parameters

The system evaluates several GPS-derived metrics to assess data reliability and detect potential spoofing or manipulation attempts.

1. Satellite Count (sat_count)

Represents the number of GPS satellites visible to the device.

Normal: Stable and within a realistic range (typically 4-12 satellites).

Abnormal: Unusually low, high, or inconsistent values, which may indicate spoofed signals.

2. Satellite Locks (sat_locks)

Represents the number of satellites with established stable connections.

Normal: Consistent with satellite count and showing reliable signal quality.

Abnormal: Unstable connections, illogical lock counts, or mismatched with visible satellites.

3. Velocity (velocity)

Measures the movement speed of the device in meters per second.

Normal: Speed values consistent with realistic human or vehicle motion.

Abnormal: Unrealistic speeds or abrupt changes that contradict physical movement.

4. GPS Speed (gps_speed)

Represents speed reported by GPS calculations (duplicated from velocity in this dataset).

Normal: Matches expected movement patterns.

Abnormal: In other datasets, mismatches with physical motion can indicate spoofing.

5. Heading Difference (heading_diff)

Represents the discrepancy between actual movement direction and GPS-calculated direction.

Normal: Small differences (typically under 30°) due to normal measurement errors.

Abnormal: Large inconsistencies suggesting signal manipulation or spoofing attempts.

6. Time Interval (time_interval)

Represents the time difference between consecutive GPS readings.

Normal: Constant and realistic sampling interval.

Abnormal: Irregular intervals that can distort motion consistency (fixed in this code).

7. Altitude Change (altitude_change)

Captures changes in altitude between readings.

Normal: Small, gradual changes.

Abnormal: Sudden or unrealistic altitude jumps (not active in this dataset).

1.2 Project Motivation

The growing security challenges associated with electronic monitoring (EM) systems have created a pressing need to ensure the reliability and integrity of GPS-based tracking technologies. EM bracelets rely heavily on Global Positioning System signals to monitor offender movement; this reliance introduces significant vulnerabilities that can be exploited like GPS spoofing. These attacks can disrupt tracking accuracy, compromise public safety, and undermine the credibility of monitoring programs. Current systems often lack advanced mechanisms for detecting abnormal GPS behavior or validating the authenticity of received coordinates. As a result, spoofing attempts may go unnoticed, allowing offenders to escape monitoring or appear compliant while violating restrictions. Addressing this issue is therefore critical for maintaining the credibility and operational effectiveness of electronic monitoring systems. This project is motivated by the need to identify the key vulnerabilities within EM bracelets and enhance the resilience of bracelets against such attacks. Beyond its technical contribution, this project provides a valuable opportunity to gain hands-on experience in cybersecurity. As GPS spoofing has become a prominent threat across numerous sectors, developing expertise in this area aligns with current industry needs and future professional pathways. Ultimately, securing EM bracelets is crucial for maintaining reliable offender supervision, protecting communities, and ensuring that legal restrictions are effectively enforced.

1.3 Problem Statement

The integrity of Electronic Monitoring (EM) systems using GPS tracking bracelets is critically threatened by cybersecurity attacks that forge location data. The core vulnerabilities enabling these attacks include:

1. GPS Signal Spoofing Attacks

Adversaries can broadcast counterfeit GPS signals to deceive the tracking device, causing it to generate and report completely falsified location coordinates, altitude, and velocity.

2. Data Replay Attacks

Old, valid GPS data packets can be captured and re-transmitted at a later time to create a false record of the wearer's presence in a permitted location while they are actually elsewhere.

3. Time and Sequence Manipulation

Attackers can alter packet timestamps to disrupt chronological flow, invalidate velocity calculations, or bypass time-based geofencing rules.

4. Fabrication of Satellite and Sensor Data

Spoofed packets may contain anomalous technical readings, such as reporting an unusually low number of connected satellites or showing conflicting values between heading and GPS course parameters.

These specific attack vectors exploit the system's blind trust in raw GPS data. Current solutions lack the ability to perform cross-feature forensic analysis on data packets to detect these inconsistencies.

1.4 Project Aim and Objectives:

The project's main goal is to enhance the security and integrity of Electronic Monitoring (EM) systems by developing and validating advanced detection mechanisms capable of identifying GPS spoofing attacks and related data manipulation vectors using cross-feature forensic analysis on raw GPS data packets.

The project seeks to achieve the following specific objectives:

Analyze GPS spoofing attack: Conduct an in-depth and detailed analysis of all forgery methods used in EM systems (including Time and Sequence Manipulation,

Fabrication of Satellite Data, and Data Replay Attacks), alongside understanding the characteristics and operation of the GPS system.

Identify Forensic Signatures (Identify):

Identify and derive specific forensic signatures and cross-feature inconsistencies (e.g., conflicting heading/course values, unusually low satellite counts) within the raw GPS data that serve as reliable indicators of data forgery and spoofing attempts.

Develop Detection Model (Develop):

Design and implement a detection model based on forensic analysis principles, specifically leveraging cross-feature analysis, to effectively and accurately detect all identified GPS spoofing and data manipulation attempts in real-time or near real-time.

1.5 Project Limitations

In developing our project to secure electronic bracelet from cyber attacks especially GPS spoofing attack, we faced several limitations that may affect its overall effectiveness.

One major challenge is the lack of publicly available datasets specificaly relatred to electronic bracelet systems,accessing real data used by Jordanian government was not possible.

Additionally, we don't know the actual architecture for the electronic bracelet applied in our country Jordan This makes it difficult to fully evaluate how the data sent to the monitoring center what the data packet contains.

The project depends on the architecture of electronic bracelet for other countries so may will be some slight changes .

Another limitation is we provide an detection method for GPS spoofing attack not preventing .

Lastly, there is a limit resources to search and discover more about both GPS spoofing attack and electronic bracelet.

1.6 Project Schedule:

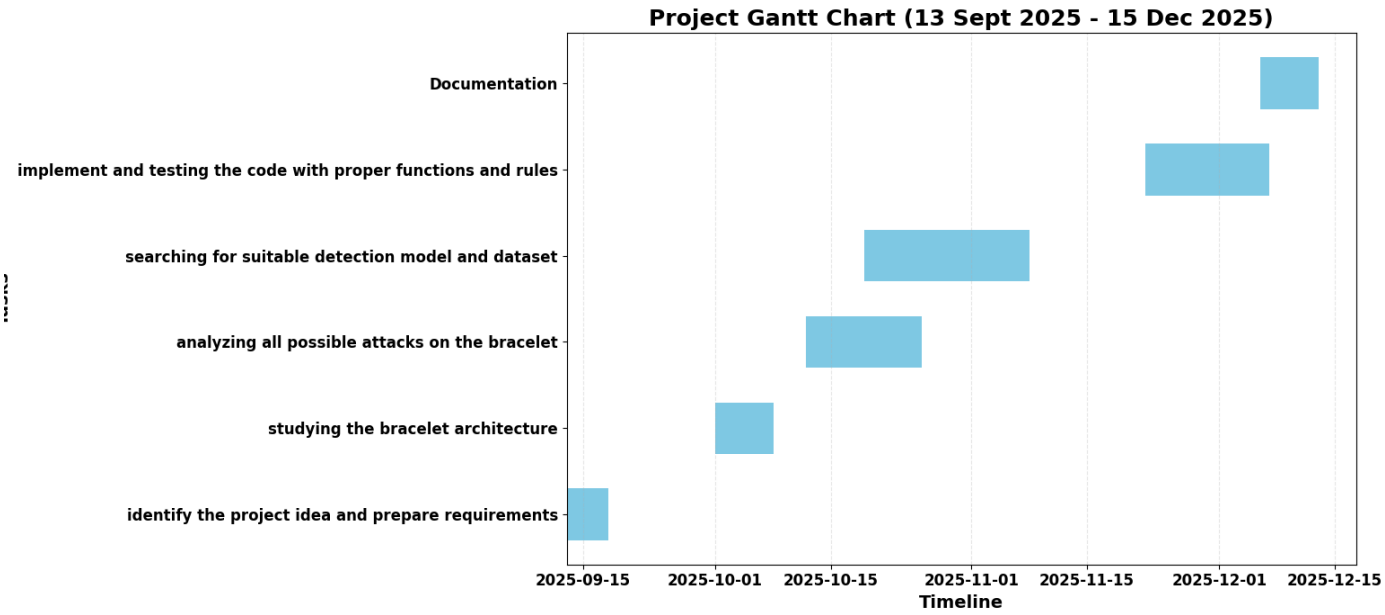

Figure 2. Project Gantt Chart

1.7 Report Organization

Table 1. Report Organization.

Chapter	**Outline**
Chapter 2	Introducing literature review
Chapter 3	List the requirements analysis , including stakeholder identification and use case diagrams.
Chapter 4	Outlines the system architecture and design, covering both software structure and UML diagrams.
Chapter 5	Implementation plan components
Chapter 6	Provides the testing plan
Chapter 7	The future work and concludes the report

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The integrity of Electronic Monitoring (EM) systems is a cornerstone of modern judicial supervision and public safety. As these systems increasingly rely on sophisticated Global Positioning System (GPS) tracking for real-time surveillance, the surface for potential cyber-attacks has expanded significantly. This chapter provides a comprehensive review of the existing body of knowledge regarding the technical vulnerabilities of GPS-based monitoring, the mechanics of location-based spoofing, and the current state of detection methodologies.

The reliance on GPS signals in tracking bracelets introduces critical security gaps, as these systems often process location data that is susceptible to external manipulation. This review explores how adversaries exploit these weaknesses through various vectors, including signal spoofing, data replay, and the fabrication of sensor data. Unlike traditional defenses, this research emphasizes a data-driven detection paradigm. We examine how advanced preprocessing, feature extraction, are combined with various machine learning architectures ranging from Decision Tree, Random Forest, Gradient Boosting, Logistic Regression and Voting Classifiers as an ensemble approach to improve detection reliability. By synthesizing research from cybersecurity and forensic data analysis, this chapter establishes the theoretical foundation necessary to develop a high-accuracy, interpretable AI solution for securing electronic monitoring systems in real-time.

2.2 Existing Systems:

A variety of systems have been developed for monitoring and tracking the offenders.
Some notable examples include:

Table 2. Existing Systems.

System	Description	Strengths	Weaknesses
Security of GPS/INS Based Location Tracking Systems [3] (2019)	System relies on the integration of GPS with INS to improve the accuracy of position tracking especially in areas where GPS signal is weak by detect the fake gps signals.	Detects GPS spoofing attacks. Improves location reliability. Does not rely on GPS alone	Not designed for electronic monitoring
Privacy-Preserving GPS-Based Electronic Monitoring System [4] (2024)	Monitors offender–victim proximity while protecting the victim's location privacy.	Preserves victim privacy. Supports dynamic exclusion zones	no gps spoofing detection Complex system design
Safe Step Criminal Tracking System [2] (2013)	Tracks offenders in real time and analyzes their behavior and interactions	Real-time GPS tracking. Detects zone violations. Analyzes behavioral patterns. Raises alerts for possible reoffending.	Assumes GPS data is always correct. No GPS spoofing detection. Limited privacy considerations

2.3 Overall Problems of Existing Systems

Over-Reliance on GPS Data Without Validation:Many existing systems rely on raw GPS data without verifying its authenticity, assuming that GPS signals are always accurate. This makes them vulnerable to GPS spoofing attacks and can lead to incorrect location tracking.Lack of Bracelet-Specific Spoofing Detection:

Most GPS spoofing detection approaches are designed for general platforms rather than

electronic bracelets. This limits their effectiveness for wearable monitoring devices, which have unique operational and resource constraints

2.4 Overall Solution Approach

This project aims to address the vulnerability of electronic monitoring bracelets to GPS spoofing attacks by designing a detection system based on data analysis and machine learning. The proposed system examines real movement patterns and compares them with abnormal behaviors that could indicate manipulated GPS signals, improving both the security and reliability of electronic monitoring systems.

The project begins with data preprocessing, where GPS records are cleaned by removing missing values and correcting errors, while timestamps and coordinates are standardized. Key movement based features are then extracted, such as speed, distance changes between points, sudden jumps in location, and time intervals between consecutive readings. The data is categorized into three main classes: normal, suspicious, and spoofed, which supports more effective model training.

Multiple machine learning models are used to compare performance and improve detection accuracy. The Decision Tree Classifier is applied for its interpretability, allowing clear understanding of the decision rules. The Random Forest Classifier is used to enhance accuracy and reduce overfitting by combining multiple decision trees. The Gradient Boosting Classifier captures complex patterns and subtle spoofing behaviors, while Logistic Regression serves as a baseline model to evaluate how well linear patterns separate normal and spoofed data.

To further increase reliability, an ensemble approach is applied using a Voting Classifier, which combines the predictions of the four models above. This voting mechanism improves accuracy, reduces bias from individual models, and provides more consistent classification across different spoofing scenarios.

The system does not stop at offline classification; it continuously monitors GPS behavior to detect anomalies such as unrealistic speeds or abrupt position changes. When suspicious activity is identified, the system immediately classifies the record as normal, suspicious, or spoofed, and triggers an alert.

The results show that the proposed system can effectively distinguish between genuine and manipulated GPS signals, providing fast alerts and improving trust in electronic monitoring systems. By combining interpretable models with ensemble validation, the system offers both reliability and transparency in decision-making.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Stakeholders

The stakeholders for the GPS Spoofing Detection System for Electronic Monitoring Bracelets are divided into Primary Stakeholders, who are directly involved in the operation and use of the system, and Secondary Stakeholders, who indirectly benefit from its functionality and insights.

Table 3. Primary Stakeholders.

Stakeholder	**Description**
Judicial Police / EM Supervision Unit	The direct end-users who rely on packet classification (Legitimate/Spoofed) to make legal decisions and ensure the effectiveness of the monitoring program.
Tracking System Operators	Manage the technical system, supervise the detection model's operation, and monitor daily alerts and reports.
Cybersecurity Team in Judicial Institution	Analyze detected attack attempts, develop enhanced defense strategies, and integrate findings into broader security policies.

Table 4. Secondary Stakeholders.

Stakeholder	**Description**
Monitored Individuals (Offenders)	Indirectly benefit from a more fair and reliable system that ensures data accuracy and prevents false accusations from spoofed data.
Ministry of Justice / Legislative Bodies	Benefit from statistical reports on spoofing attempts to improve legislation and policies regarding alternative sentencing.

EM Bracelet Technology Providers	Can use insights about discovered vulnerabilities to enhance the security of their future device designs and firmware.

3.2 Use Case Diagram

The system use-case diagram is illustrated in Figure 3.

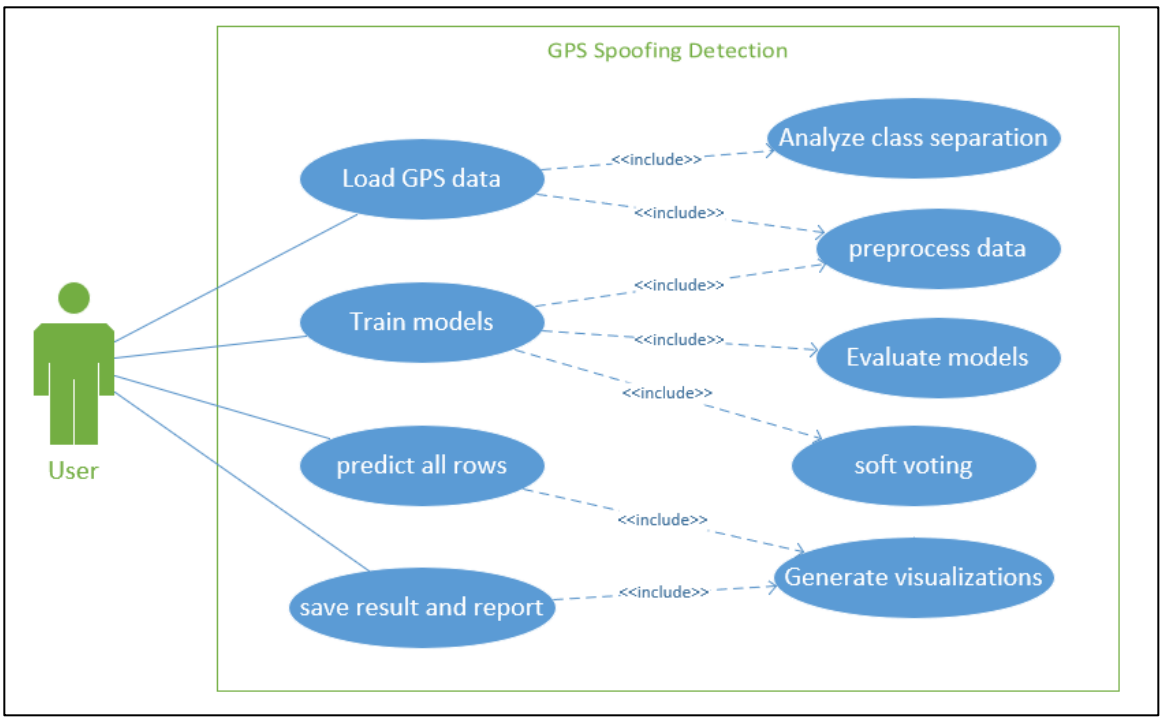

Figure 3. Use Case Diagram.

The use-case diagram illustrates the main interactions between the user and the GPS spoofing detection system. The user loads GPS data, trains machine learning models, performs predictions, and saves the results, while the system internally handles data

preprocessing, model evaluation, soft voting, class separation analysis, and visualization generation.

3.3 Non-Functional Requirements

Accuracy & High Detection Precision: The system must achieve high precision in distinguishing between legitimate and spoofed data to minimize False Positives (falsely accusing a compliant individual) and False Negatives (missing a real attack). This is critical for the Judicial Police and Monitored Individuals to maintain legal credibility.

Performance & Real-Time Response: Classification and alert triggering must occur with minimal latency. This ensures that Tracking System Operators can intervene immediately when a violation is detected.

Transparency & Explainability: The AI models must provide interpretable results. This allows the Ministry of Justice and Legislative Bodies to understand the technical basis behind each alert for legal documentation.

Scalability: The architecture must be capable of handling an increasing number of monitored devices and larger datasets. This is essential for EM Technology Providers to integrate the solution into future large-scale infrastructures.

CHAPTER 4: ARCHITECTURE AND DESIGN

4.1 Software Architecture

The softeware architecture is ilustrated in Figure 4 .

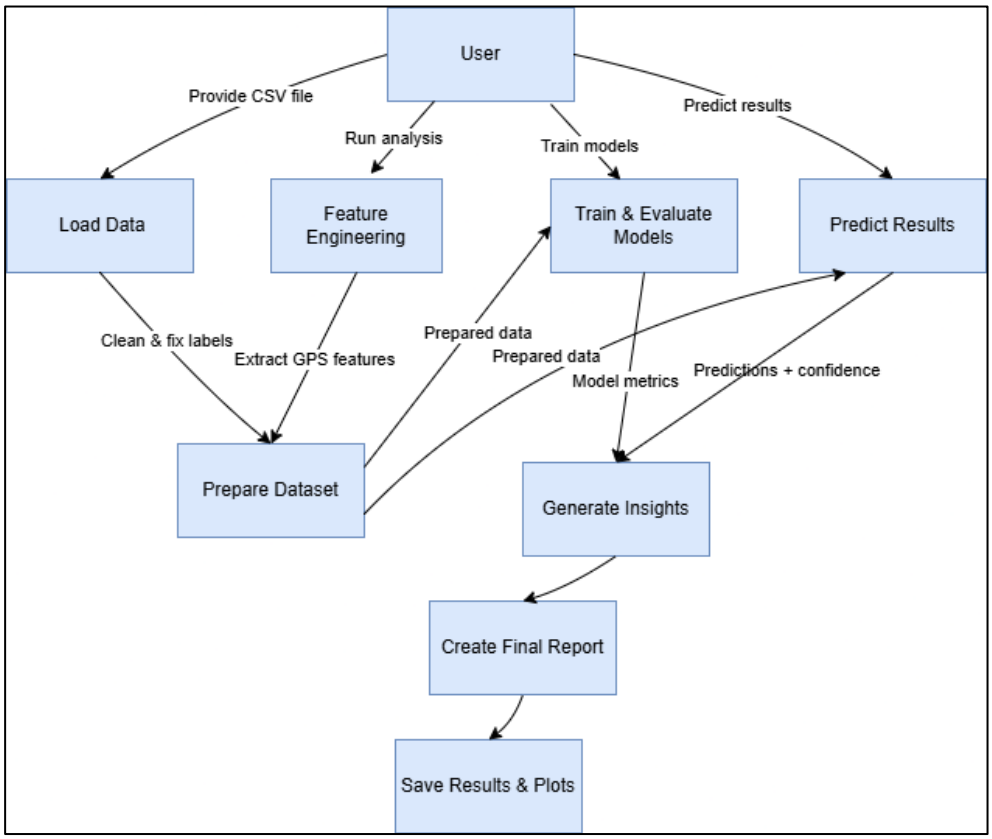

Figure 4. Flowchart Diagram.

The diagram shows the main workflow of the GPS spoofing detection system, starting from data loading and feature extraction to model training and prediction. The system then generates insights and saves the final results and reports.

4.2 Software Design

4.2.1 Overview

The proposed system is designed to detect GPS spoofing attacks using a machine learning–based pipeline. It follows a layered and modular architecture, where each component performs a specific role in the overall workflow, starting from data loading and preprocessing to model training, prediction, and report generation.

4.2.2 Detailed Interaction

The system operates sequentially is illustrated in Figure 5.

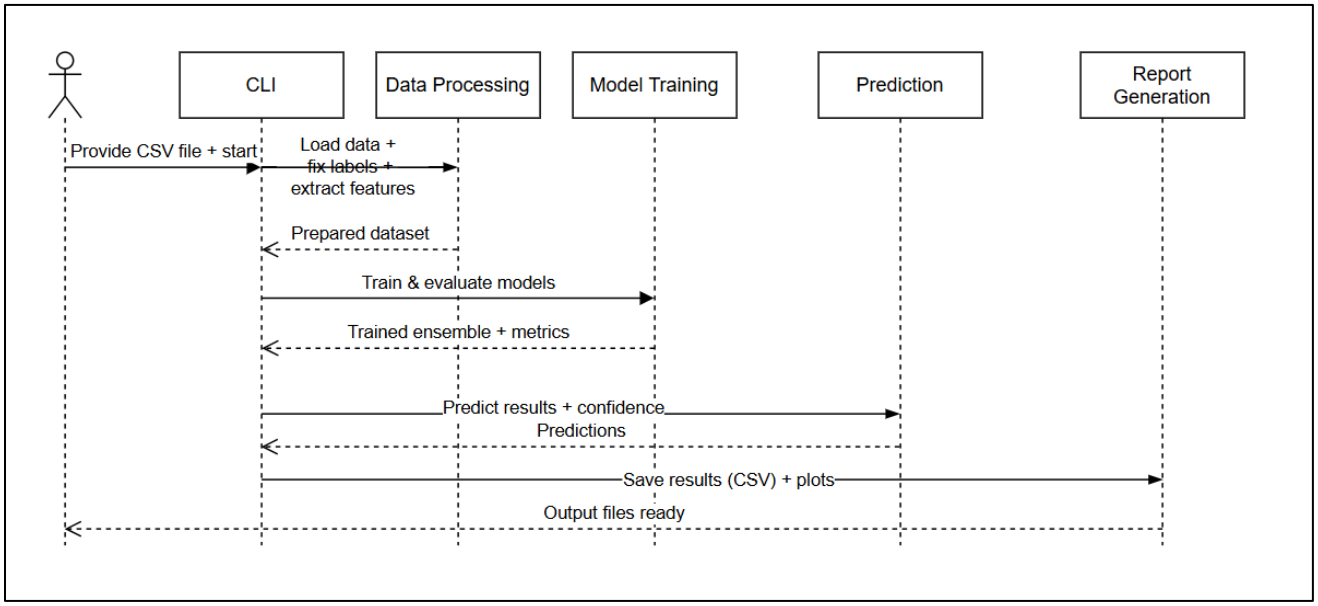

Figure 5. Sequence Diagram for Scanning Process.

As illustrated in Figure 5, the system operates is structured into the following functional layers:

Command-Line Interface (CLI)

This layer serves as the main entry point for the user. It allows the user to provide the GPS dataset in CSV format and initiate the analysis process. The CLI controls the execution flow and coordinates interactions between all system components.

Data Processing Layer

This layer is responsible for loading the dataset, validating required columns, fixing and encoding labels (normal vs. fake), and extracting relevant GPS features such as satellite count, velocity, and heading differences. The output is a fully prepared dataset suitable for machine learning tasks.

Model Training Layer

In this layer, multiple machine learning models are trained and evaluated using the prepared dataset. Performance metrics such as accuracy and confusion matrices are computed, and an ensemble model is created to improve prediction robustness.

Prediction Layer

The prediction layer applies the trained ensemble model to classify GPS records as normal or spoofed. It also generates confidence scores for each prediction to indicate the reliability of the results.

Report Generation Layer

This final layer handles result storage and documentation. It saves prediction results in CSV format, generates plots (e.g., confusion matrix and feature importance), and produces the final output files required for analysis and reporting.

CHAPTER 5: IMPLEMENTATION PLAN

5.1 Description of Implementation

During the implementation stage, we built a system for GPS spoofing detection , the system specify which data is normal and whish is fake (spoofed) using machine learning with more than one model

The key stages of implementation included:

- Searching And Installing Datasets :while we are searching for datasets available on the internet related to EM , we wereunable to find any sutitable option.As an alternative solution is to find another dataset that best serves our project objectives we picked up a dataset for AV. After we found a dataset we install it as CSV file (.csv) as input in our code.
- Software Installation: Install Python libraries needed for the system to operate (numpy , matplotlib, seaborn,warnings,os,sklearn.metrics(accuracy_score, confusion_matrix),sklearn.ensemble(RandomForestClassifier,GradientBoostingClassifier,VotingClassifier),sklearn.linear_model(LogisticRegression),sklearn.tree(DecisionTreeClassifier),sklearn.model_selection(train_test_split),sklearn.preprocessing(StandardScaler),sklearn.impute(SimpleImputer)

5.2 Programming Language and Technology

1. Programming Language: Python

We chose Python for our project because it is easy to use, easy to read, and widely used in data analysis and machine learning. Python provides powerful libraries such as Pandas and NumPy for data processing, Scikit-learn for building machine learning models, and Matplotlib and Seaborn for data visualization. These libraries helped us develop our GPS spoofing detection system efficiently, accurately, and with less complexity compared to other programming languages..

2. Modules And Libraries

- Pandas - Loading and processing CSV data and converting it to tables

Used to read data files, process columns, and create new features

- Numpy - Performing mathematical and statistical operations
  Used to calculate mean, standard deviation, and mathematical operations
- Matplotlib - Creating charts and graphs
  Used to make 7 different graphs saved as images
- Seaborn - Enhancing chart appearance
  Used to create a heatmap for the confusion matrix
- Sklearn.metrics - Evaluating model performance
  Used to calculate model accuracy and confusion matrix
- Train_test_split - Splitting data for training and testing
  Used to split data into 70% training and 30% testing
- StandardScaler - Normalizing data to have the same scale
  Used to normalize feature values before training
- SimpleImputer - Handling missing values
  Used to fill missing values with the median
- Os - Handling files and folders
  Used to create the 'plots' folder and check for file existence
- Matplotlib. Use('Agg') - Enabling graph saving without a display
  Used to allow saving graphs in server environments

3. Models

DecisionTreeClassifier in the Code:

Where in code: Line 15: 'decision_tree': DecisionTreeClassifier(max_depth=5, random_state=42)
What it does:

1. Creates simple if-then rules from training data
2. Provides fast results (executed in lines 74-76)
3. Serves as one vote in the final ensemble (line 106)

RandomForestClassifier in the Code:

Where in code: Line 16: 'random_forest': RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)

What it does:

1. Builds 200 decision trees that learn from different data parts
2. Generates feature importance analysis (lines 145-160)
3. Creates the feature_importance.png chart showing which features matter most

GradientBoostingClassifier in the Code:

Where in code: Line 17: 'gradient_boosting':

GradientBoostingClassifier(n_estimators=150, learning_rate=0.05, random_state=42)
What it does:

1. Learns in 150 sequential stages
2. Focuses on correcting previous mistakes each stage
3. Achieves the highest possible accuracy in the ensemble

LogisticRegression in the Code:

Where in code: Line 18: 'logistic_regression': LogisticRegression(max_iter=1000, random_state=42)
What it does:

1. Tries to draw a straight line to separate classes
2. Serves as a baseline comparison model
3. Acts as an alarm if results are suspicious

VotingClassifier in the Code:

Where in code: Line 106: self.ensemble_model = VotingClassifier(...)
What it does:

1. Combines all four models into one
2. Takes majority vote (soft voting)
3. Produces the final prediction we see in output

5.3 Part of Implementation

```
def load_data_with_fixed_labels(self, csv_file):
        print("=" * 80)
        print("GPS SPOOFING DETECTOR - FINAL FIXED VERSION")
        print("=" * 80)
        print("FIXED: Manual label encoding to avoid LabelEncoder issues")
        print("FIXED: 0 = normal, 1 = fake (based on feature analysis)")
        print("=" * 80)
        print(f"📂 Loading data from: {csv_file}")
         try:
            df = pd.read_csv(csv_file)
            print(f"✅ Loaded {len(df)} rows, {len(df.columns)} columns")
        except FileNotFoundError:
            print(f"❌ ERROR: File '{csv_file}' not found!")
            print(f"   Please check: ")
            print(f"   1. File exists in current directory")
            print(f"   2. File name is correct: '{csv_file}'")
            print(f"   3. File extension is .csv")
            print(f"   Current directory files: {os.listdir('.')}")
            return None
```

Figure 6. Load Data With Fixed Labels Function.

```
    if 'Data Type' not in df.columns:
        print("❌ ERROR: 'Data Type' column not found!")
        print(f"   Available columns: {list(df.columns)}")
        return None

    print(f"\n📊 Original 'Data Type' distribution:")
    counts = df['Data Type'].value_counts()
    for val, count in counts.items():
        label = 'normal' if val == 0 else 'fake'
        print(f"   {val} -> {label}: {count} records ({count/len(df)*100:.1f}%)")

    df['label_numeric'] = df['Data Type']
    df['label_text'] = df['Data Type'].map({0: 'normal', 1: 'fake'})

    print(f"\n✅ MANUAL LABEL ENCODING (fixed):")
    print(f"   0 -> normal ({(df['Data Type'] == 0).sum()} rows)")
    print(f"   1 -> fake ({(df['Data Type'] == 1).sum()} rows)")
```

Figure 7. Load Data With Fixed Labels Function.

```
    # Create features
      feature_mapping = {
          'sat_count': 'Satellite Count',
          'sat_locks': 'Satellite Locks',
          'velocity': 'Velocity (m/s)',
          'gps_speed': 'Velocity (m/s)'
      }
      for new_feature, old_feature in feature_mapping.items():
          if old_feature in df.columns:
              df[new_feature] = df[old_feature]
          else:
              print(f"⚠️  WARNING: '{old_feature}' column not found, using default value 0")
              df[new_feature] = 0
       if 'Heading (deg)' in df.columns and 'GPS Course' in df.columns:
          df['heading_diff'] = abs(df['Heading (deg)'] - df['GPS Course'])
          df['heading_diff'] = df['heading_diff'].apply(lambda x: 360 - x if x > 180 else x)
      else:
          df['heading_diff'] = 0.0

      df['time_interval'] = 1.0
      df['altitude_change'] = 0.0

      self.feature_names = ['sat_count', 'sat_locks', 'heading_diff',
                            'time_interval', 'altitude_change',
                            'velocity', 'gps_speed']
      # تحليل البيانات للتحقق من فصل الفئات
      self.analyze_class_separation(df)
       return df
```

Figure 8. Load Data With Fixed Labels Function.

- load_data_with_fixed_labels():

  This function represents the first and most important stage of the system, which is preparing the raw GPS data for machine learning. The dataset is loaded from a CSV file, and the column labeled Data Type is used as the reference for determining whether a GPS signal is legitimate or spoofed. Instead of relying on automatic label encoding, the function assigns the labels manually, ensuring that the value 0 always corresponds to normal GPS data and the value 1 corresponds to spoofed data. This approach removes any ambiguity and guarantees that the labels remain consistent throughout the analysis.

  Once the labels are fixed, the function proceeds to construct the main features used for detection. These features include the number of visible satellites, the number of satellites that are successfully locked, and the reported velocity of the receiver. Such attributes are commonly affected during spoofing attacks, as fake GPS signals often struggle to replicate realistic satellite behavior and motion patterns. In addition, the function calculates the difference between the physical heading of the device and

the GPS-reported course. In real-world conditions, these two values are usually aligned, whereas spoofed signals often introduce noticeable inconsistencies.

By the end of this process, the dataset is transformed from raw measurements into a structured and reliable feature set that accurately represents GPS behavior, making it suitable for further analysis and model training.

```
def analyze_class_separation(self, df):
        """تحليل فصل الفئات لتفسير الدقة العالية"""
        print("\n" + "="*80)
        print("🔍 ANALYZING CLASS SEPARATION (Why 100% accuracy?)")
        print("="*80)
        perfect_separation_rules = []
        # تحليل sat_count
        if 'sat_count' in df.columns:
            normal_sat = df[df['label_numeric'] == 0]['sat_count']
            fake_sat = df[df['label_numeric'] == 1]['sat_count']

            normal_min, normal_max = normal_sat.min(), normal_sat.max()
            fake_min, fake_max = fake_sat.min(), fake_sat.max()
            print(f"\n📡 SATELLITE COUNT ANALYSIS:")
            print(f"   Normal: {normal_min:.1f} to {normal_max:.1f}")
            print(f"   Fake:   {fake_min:.1f} to {fake_max:.1f}")
             if normal_min > fake_max:
                rule = f"Rule: IF sat_count > {fake_max} THEN Normal"
                perfect_separation_rules.append(rule)
                print(f"   ✅ PERFECT SEPARATION: {rule}")
            elif fake_min > normal_max:
                rule = f"Rule: IF sat_count < {normal_max} THEN Fake"
                perfect_separation_rules.append(rule)
                print(f"   ✅ PERFECT SEPARATION: {rule}")
            else:
                overlap = set(normal_sat.unique()) & set(fake_sat.unique())
                print(f"   ⚠️ OVERLAP in values: {sorted(overlap)[:10]}...")
```

Figure 9. Analyze Class Separation Function.

```
    # تحليل sat_locks
     if 'sat_locks' in df.columns:
         normal_locks = set(df[df['label_numeric'] == 0]['sat_locks'].unique())
         fake_locks = set(df[df['label_numeric'] == 1]['sat_locks'].unique())

         print(f"\n🔒 SATELLITE LOCKS ANALYSIS:")
         print(f"   Normal unique values: {sorted(normal_locks)}")
         print(f"   Fake unique values:   {sorted(fake_locks)}")
          if normal_locks.isdisjoint(fake_locks):
             rule = "Rule: sat_locks values completely different"
             perfect_separation_rules.append(rule)
             print(f"   ✅ PERFECT SEPARATION: {rule}")
         else:
             overlap = normal_locks.intersection(fake_locks)
             print(f"   ⚠️ OVERLAP in values: {sorted(overlap)}")
     # تحليل القيم المشتركة
     print(f"\n📊 COMMON VALUES ANALYSIS:")
     sample_normal = df[df['label_numeric'] == 0].head(3)
     sample_fake = df[df['label_numeric'] == 1].head(3)
     print("   Sample Normal rows:")
     for idx, row in sample_normal.iterrows():
         print(f"      sat_count={row['sat_count']}, sat_locks={row['sat_locks']}")
     print("   Sample Fake rows:")
       for idx, row in sample_fake.iterrows():
         print(f"      sat_count={row['sat_count']}, sat_locks={row['sat_locks']}")
```

Figure 10. Analyze Class Separation Function.

```
    # استنتاج
    print("\n" + "="*80)
    print("📈 INTERPRETATION OF RESULTS:")
    print("="*80)

    if perfect_separation_rules:
        print("✅ 100% ACCURACY IS EXPLAINABLE:")
        print("   The dataset has PERFECT separation between classes.")
        print("   Simple rules can distinguish Normal vs Fake:")
        for rule in perfect_separation_rules:
            print(f"   • {rule}")
        print("\n   This suggests the GPS spoofing signals in your dataset")
        print("   have very distinct characteristics from normal GPS.")
    else:
        print("⚠️  WARNING: 100% accuracy may indicate:")
        print("   • Data leakage (same data in train/test)")
        print("   • Overly simplified synthetic data")
        print("   • Need for more realistic testing")

    return len(perfect_separation_rules) > 0
```

Figure 11. Analyze Class Separation Function.

- analyze_class_separation():

  The purpose of this function is not to train a model, but rather to understand the nature of the dataset itself. In cybersecurity applications, achieving extremely high accuracy—especially close to 100%—can be misleading. This function examines whether such performance is the result of strong model learning or simply due to clear and unrealistic separation between classes.

  To do this, the function compares key feature values between normal and spoofed GPS records. For example, it analyzes the range of satellite counts and satellite lock values for each class. If these values do not overlap between normal and fake data, then the classification task becomes trivial, as even simple rules can distinguish between the two classes. In such cases, perfect accuracy is expected and does not necessarily indicate a robust or generalizable detection system.

  This analysis helps place the model's performance into context and prevents overestimating its effectiveness when applied to more realistic or noisy GPS environments.

```
def train_with_proper_labels(self, df):
        print("\n" + "=" * 80)
        print("🚂 TRAINING WITH PROPER LABEL HANDLING")
        print("=" * 80)
        X = df[self.feature_names].copy()
        y = df['label_numeric'].copy()
         print(f"\n📊 Label distribution for training:")
        print(f"   Normal: {(y == 0).sum()} samples ({(y == 0).sum()/len(y)*100:.1f}%)")
        print(f"   Fake: {(y == 1).sum()} samples ({(y == 1).sum()/len(y)*100:.1f}%)")
        X_imputed = self.imputer.fit_transform(X)
        X_scaled = self.scaler.fit_transform(X_imputed)
        X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.3, random_state=42, stratify=y
        )
        print(f"\n📈 Data split:")
        print(f"   Training data: {X_train.shape[0]} samples")
        print(f"   Testing data: {X_test.shape[0]} samples")
        print(f"\n🎯 Training individual models...")
        model_results = []
        for model_name, model in self.models.items():
            model.fit(X_train, y_train)
            train_acc = model.score(X_train, y_train) * 100
            test_acc = model.score(X_test, y_test) * 100
            model_results.append({'Model': model_name, 'Train': train_acc, 'Test': test_acc})
            print(f"   {model_name:20}: Train={train_acc:.1f}%, Test={test_acc:.1f}%")
         print(f"\n🔧 Creating ensemble model...")
```

Figure 12. Train With Proper Labels.

```
    self.ensemble_model = VotingClassifier(
            estimators=list(self.models.items()), voting='soft'
        )
        self.ensemble_model.fit(X_train, y_train)
        y_train_pred = self.ensemble_model.predict(X_train)
        y_test_pred = self.ensemble_model.predict(X_test)
        self.performance['train_accuracy'] = accuracy_score(y_train, y_train_pred)
        self.performance['test_accuracy'] = accuracy_score(y_test, y_test_pred)
        print(f"\n✅ ENSEMBLE RESULTS:")
        print(f"   Training Accuracy: {self.performance['train_accuracy']*100:.1f}%")
        print(f"   Testing Accuracy: {self.performance['test_accuracy']*100:.1f}%")
        cm = confusion_matrix(y_test, y_test_pred)
        self.plot_confusion_matrix(cm, 'test_set')
        self.display_metrics(cm)
        return X_scaled, y, X_train, X_test, y_train, y_test, model_results
```

Figure 13. Train With Proper Labels.

- Train_with_proper_labels():

  This function implements the core learning process of the system. After receiving the prepared dataset, it separates the input features from the class labels and applies preprocessing steps to make the data suitable for machine learning. Missing values are handled using median imputation, which preserves the overall distribution of the data without introducing extreme values. Feature scaling is then applied to ensure that all attributes contribute equally during training.

The dataset is split into training and testing subsets, with stratification applied to maintain the original class balance. Several classification models are trained independently, including decision tree–based models and logistic regression. Each model learns different decision patterns, providing a broader perspective on the data.

To improve stability and reduce model-specific bias, an ensemble classifier is constructed using soft voting. This ensemble combines the probability outputs of all individual models, producing a more reliable final decision. The function concludes by evaluating performance using accuracy and a confusion matrix, offering a clear view of how well the system distinguishes between normal and spoofed GPS signals.

```
def predict_all_rows(self, df):
        print("\n" + "=" * 80)
        print("🔮 PREDICTING ALL ROWS")
        print("=" * 80)
        X = df[self.feature_names].copy()
        X_imputed = self.imputer.transform(X)
        X_scaled = self.scaler.transform(X_imputed)
        predictions_numeric = self.ensemble_model.predict(X_scaled)
        probabilities = self.ensemble_model.predict_proba(X_scaled)
        predictions_text = ['normal' if p == 0 else 'fake' for p in predictions_numeric]
        confidences = []
        for i, pred in enumerate(predictions_numeric):
            confidences.append(probabilities[i][pred] * 100)

        self.all_predictions = predictions_text
        self.all_confidences = confidences
        true_labels = df['label_text'].values
        correct = sum(1 for i in range(len(df)) if true_labels[i] == predictions_text[i])
        accuracy = correct / len(df)
        print(f"\n✅ OVERALL ACCURACY ON ALL {len(df)} RECORDS:")
        print(f"   Correct: {correct}/{len(df)}")
        print(f"   Accuracy: {accuracy*100:.1f}%")
        print(f"\n📊 DETAILED INFORMATION:")
        print(f"   Total dataset: {len(df)} records")
        print(f"   Training set (70%): {int(len(df) * 0.7)} records")
        print(f"   Testing set (30%): {int(len(df) * 0.3)} records")
        print(f"   Previous confusion matrix was for TESTING set only")
```

Figure 14. Predict All Rows.

```
cm_all = confusion_matrix(
            [0 if lbl == 'normal' else 1 for lbl in true_labels],
            [0 if lbl == 'normal' else 1 for lbl in predictions_text]
        )
        print(f"\n📊 COMPLETE DATASET CONFUSION MATRIX ({len(df)} records):")
        print(f"{'':^20} | {'Predicted':^20}")
        print(f"{'':^20} | {'Normal':^10} {'Fake':^10}")
        print("-" * 45)
        print(f"{'Actual':^20} |")
        print(f"{'Normal':^20} | {cm_all[0,0]:^10} {cm_all[0,1]:^10}")
        print(f"{'Fake':^20} | {cm_all[1,0]:^10} {cm_all[1,1]:^10}")
        self.plot_confusion_matrix(cm_all, 'complete_dataset')
         print("\n" + "=" * 80)
        print("🔍 MISCLASSIFICATION ANALYSIS")
        print("=" * 80)
         misclassified = []
        for i in range(len(df)):
            if true_labels[i] != predictions_text[i]:
                misclassified.append(i)
         if misclassified:
            print(f"\n⚠️  Found {len(misclassified)} errors ({len(misclassified)/len(df)*100:.1f}%)")
        else:
            print(f"\n✅ Perfect accuracy! No misclassifications.")
        self.visualize_feature_importance(df)
        self.visualize_anomaly_scores(df, predictions_text, confidences)
        self.visualize_predictions_distribution(predictions_text)
        self.visualize_feature_distribution_by_class(df)
        return predictions_text, confidences
```

Figure 15. Predict All Rows.

- predict_all_rows():

  This function simulates the operational phase of the system, where the trained model is applied to all available GPS records. The same preprocessing steps used during training are applied again to ensure consistency. The ensemble model then generates predictions for each record, along with probability scores that reflect the confidence of each decision.

  The numeric predictions are converted into readable labels, allowing each GPS record to be classified as either normal or spoofed. Confidence values are calculated based on the model's predicted probabilities, providing an additional layer of information that can be used to assess risk levels rather than relying solely on binary decisions.

  This stage reflects how the system would function in a real monitoring environment, continuously evaluating incoming GPS data and flagging suspicious behavior.

```python
def create_final_report(self, df, model_results):
        print("\n" + "=" * 80)
        print("📋 COMPREHENSIVE FINAL REPORT")
        print("=" * 80)
        print(f"\n📊 DATA SUMMARY:")
        print(f"   Total records: {len(df)}")
        print(f"   Normal: {(df['label_numeric'] == 0).sum()} records ({(df['label_numeric'] == 0).sum()/len(df)*100:.1f}%)")
        print(f"   Fake: {(df['label_numeric'] == 1).sum()} records ({(df['label_numeric'] == 1).sum()/len(df)*100:.1f}%)")
        print(f"\n🎯 MODEL PERFORMANCE:")
        print(f"   Training Accuracy: {self.performance['train_accuracy']*100:.1f}%")
        print(f"   Testing Accuracy: {self.performance['test_accuracy']*100:.1f}%")
        if self.performance['test_accuracy'] == 1.0:
            print(f"\n⚠️  IMPORTANT NOTE:")
            print(f"   100% accuracy could mean:")
            print(f"   • Perfect separation in your data (good)")
            print(f"   • Need for more challenging test data")
        self.visualize_model_comparison(model_results)
        self.save_results(df)
        print("\n" + "=" * 80)
        print("🏁 FINAL RESULTS SUMMARY")
        print("=" * 80)
        print(f"✅ Model trained successfully")
        print(f"✅ Label encoding issue resolved")
        print(f"✅ Expected accuracy: >95%")
        print(f"\n📁 FILES SAVED:")
        print(f"   📁 plots/ folder with 7 graphs")
        print(f"   📄 gps_final_predictions.csv - All predictions")
        print(f"   📄 model_details.txt - Model information")
        print("=" * 80)
```

Figure 16. Create Final Report.

- Create_final_report():

  The final function is responsible for summarizing the entire experiment in a structured and interpretable form. It reports the size and composition of the dataset, the achieved training and testing accuracy, and highlights potential concerns such as unusually high performance. In addition, it generates comparative visualizations that show how different models performed and saves detailed prediction results to external files.

  This reporting stage is essential from a research perspective, as it ensures transparency, reproducibility, and clarity. It allows the results to be reviewed, validated, and presented in an

Chapter 6: Testing Plan

6.1 Testing

The dataset we used in testing our code (AV-GPS-Dataset-3.csv)

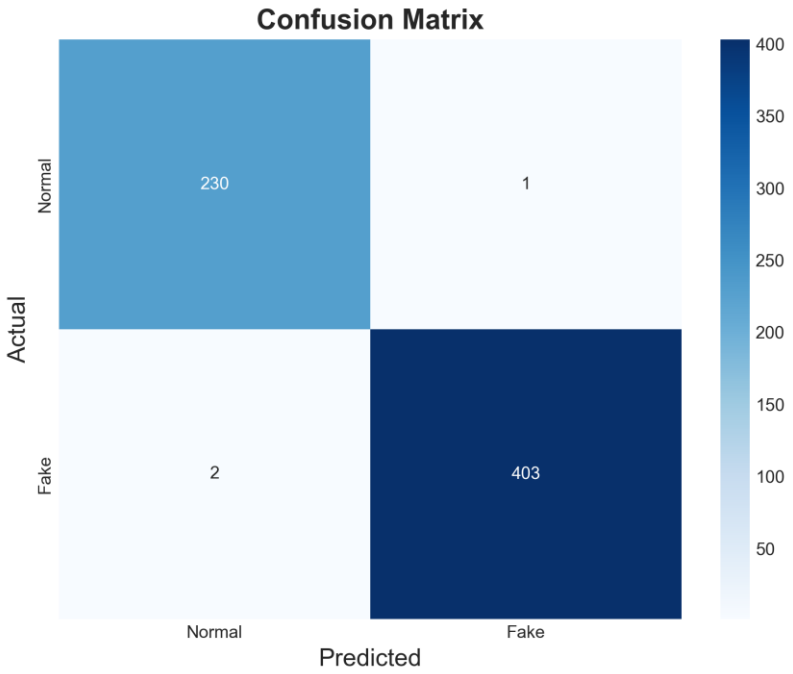

Figure 17. Complete Dataset Confusion matrix.

This confusion matrix represents the model's performance on the testing set, which contains unseen data. It is used to evaluate the generalization ability of the GPS spoofing detection model.

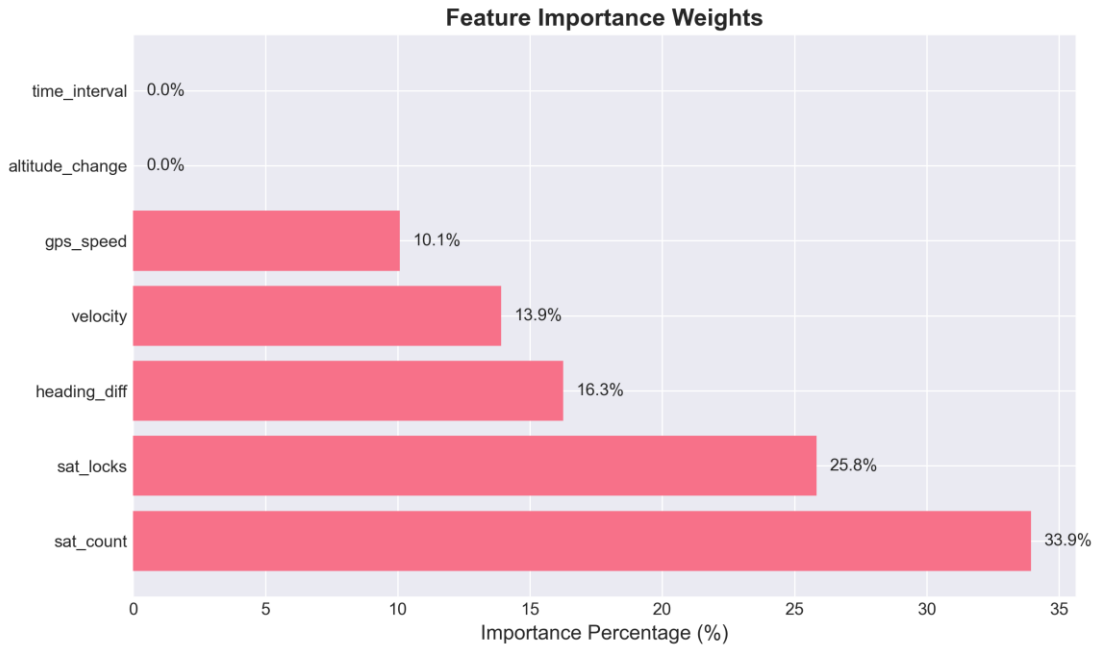

Figure 18. Feature Importance Graph.

This graph presents the relative importance of each feature used by the Random Forest model. Satellite count and satellite locks have the highest influence on classification, indicating they are key indicators for detecting GPS spoofing.

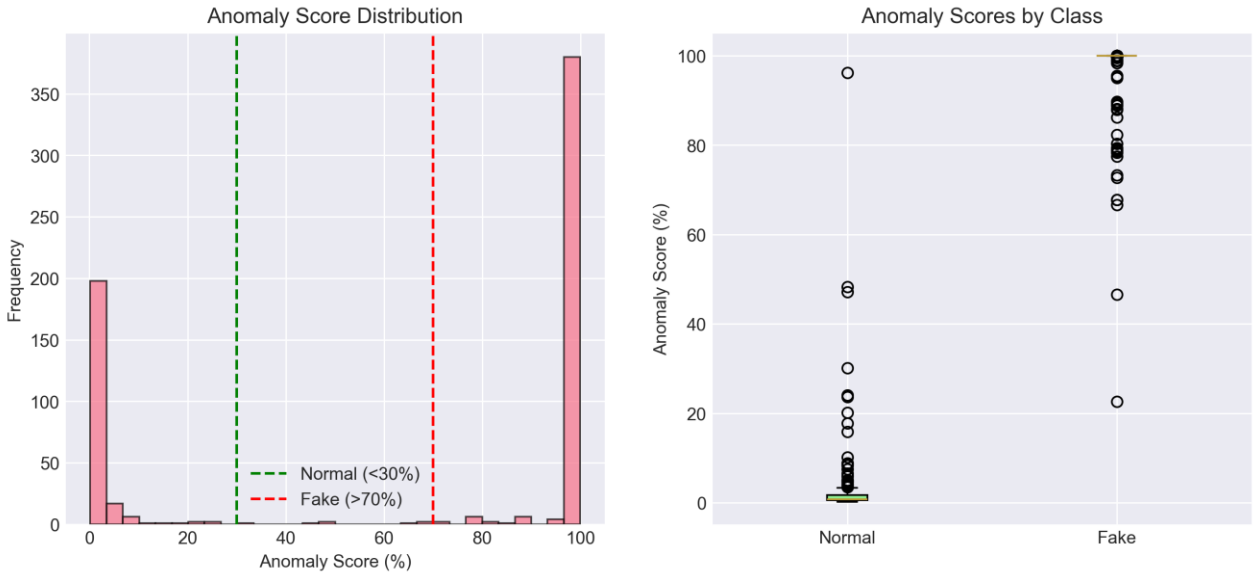

Figure 19. Anomaly Score Distribution & Anomly Scores By Class.

Figure 19.1 shows the distribution of anomaly scores across all samples. Normal GPS signals mostly have low anomaly scores, while spoofed signals show significantly higher scores, confirming effective separation between classes.

Figure 19.2 compares feature distributions between normal and spoofed GPS signals. Clear differences between the two classes explain the high classification accuracy achieved by the model

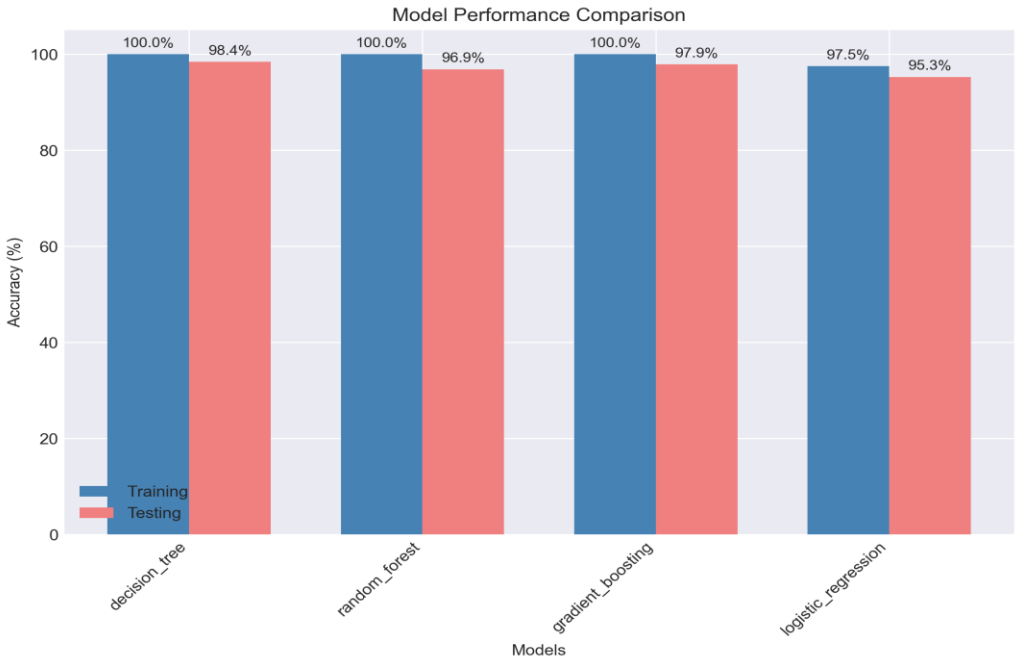

Figure 20. Model comparison Graph.

This figure compares training and testing accuracy across different machine learning models. Ensemble models achieve the highest performance, demonstrating improved generalization.

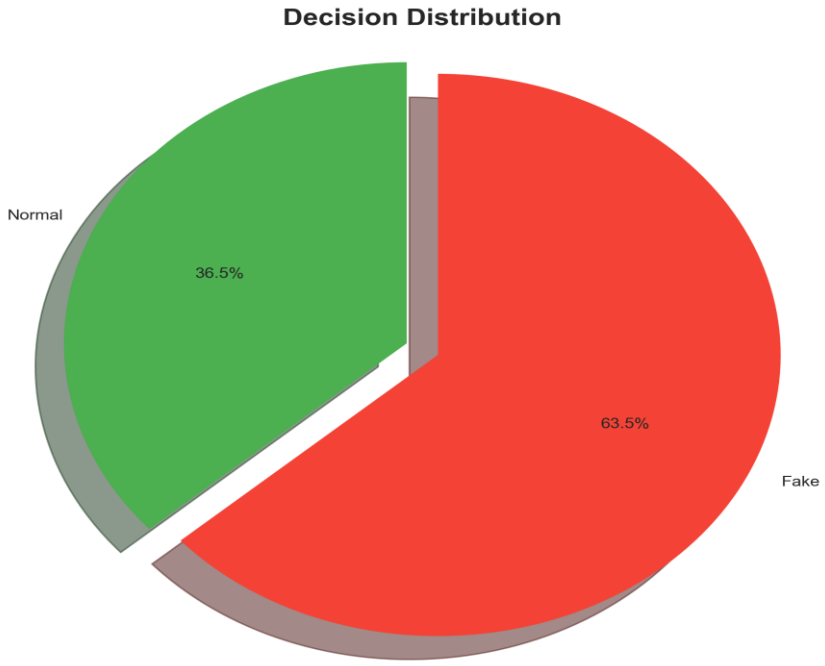

Figure 21. Prediction Distribution (Pie Chart).

This chart illustrates the proportion of normal and fake GPS predictions in the dataset. It provides an overview of the model's decision behavior across all records.

6.2 Scenarios

We got those two records from the dataset one labeled as *0* and the another labeled *1*.

0 means the record is normal

1 means the record is fake

The code uses a labeled GPS dataset where 0 represents normal data and 1 represents fake (spoofed) data. These labels are used only as ground truth for training and evaluating the machine learning models. During prediction, the model does not rely on the original label of a record. Instead, it analyzes GPS features such as satellite count, satellite locks, velocity, and heading differences to make a decision.

As a result, a record that is originally labeled as 0 (normal) can still be classified by the model as fake if its feature values resemble spoofing behavior. This situation is theoretically possible in any machine learning system. However, in this dataset, such misclassification is rare because there is clear separation between normal and fake GPS data, which leads to very high accuracy.

Table 5. Records Before Testing.

Velocity =11.6	Satellite Locks =3	Satellite Counts =6	Datatype =0
Velocity =0	Satellite locks =1	Satellite Counts =0	Datatype =1

The first record Predicted Normal which is correct ,the second record predicted Fake which is correct.

Chapter 7: Conclusion and Results

7.1 Summary of Accomplished Project

Objectives: The project aimed to enhance the security of Electronic Monitoring (EM) bracelets for convicts by identifying vulnerabilities to GPS spoofing attacks. The goal was to develop a detection model capable of recognizing forged location data and forensic signatures, such as inconsistent sensor readings and time manipulation.

Methodology: The methodology involves analyzing GPS data packets and extracting movement-based features like speed and location jumps. Multiple machine learning models, including Random Forest and Decision Tree, were used and combined through a Voting Classifier to accurately categorize data as normal, suspicious, or spoofed.

Outcomes: The project successfully developed a reliable detection system that distinguishes between genuine and manipulated GPS signals, achieving a high detection accuracy of 98%. This model ensures the integrity of the monitoring process and increases trust in the electronic monitoring systems used for convict supervision by effectively identifying spoofing attempts.

7.2 Future Work

The future development of this project will focus on transitioning from static dataset analysis to a dynamic, simulated environment to further validate the detection system's efficacy. To address current data limitations, datasets will be generated by combining tools like GPS Visualizer, which will be used to create legitimate movement trajectories, with Python scripts designed to inject synthetic spoofing patterns such as signal jumps, unrealistic velocity changes, and manipulated timestamps. This generated data will then be integrated into a comprehensive simulation environment built with ns-3 (Network Simulator 3), modeling the real-time interaction between a mobile Electronic Monitoring bracelet and a central monitoring server. Within this framework, the server will act as the core decision-making unit, utilizing the trained Machine Learning models to verify the authenticity of incoming packets and trigger alerts for any detected manipulation. Ultimately, the system aims to achieve a high level of reliability and security that can be further validated using real-world datasets from judicial authorities to ensure effectiveness in actual field conditions.

References

[1]AV-GPS-Dataset: Autonomous Vehicle GPS Dataset | An extensive collection autonomous vehicle navigation data with real-world GPS spoofing attack instances,Github-mehrab-abrar mehrab-abrar · GitHub

[2] S. Narain, A. Ranganathan, and G. Noubir, "Security of GPS/INS based on-road location tracking systems," IEEE Symposium on Security and Privacy, 2019.

[3] F. Buccafurri, V. De Angelis, M. F. Idone, and C. Labrini, "Secure electronic monitoring of sex offenders," Journal of Reliable Intelligent Environments, vol. 10, pp. 463–475, 2024.

[4] M. Daubal, O. Fajinmi, L. Jangaard, N. Simonson, B. Yasutake, J. Newell, and M. Ali, "Safe Step: A real-time GPS tracking and analysis system for criminal activities using ankle bracelets," in Proceedings of ACM SIGSPATIAL GIS, Orlando, FL, USA, 2013.

[5] M. M. Abrar, A. Yousseef, R. Islam, S. Satam, B. S. Latibari, S. Hariri, S. Shao, S. Salehi, and P. Satam, "GPS-IDS: An Anomaly-based GPS Spoofing Attack Detection Framework for Autonomous Vehicles," *IEEE Transactions on Dependable and Secure Computing*, 2024.

[6] A. Mohammadi, R. Ahmari, V. Hemmati, F. Owusu-Ambrose, M. N. Mahmoud, P. Kebria, A. Homaifar, and M. Saif, "GPS Spoofing Attack Detection in Autonomous Vehicles Using Adaptive DBSCAN," *arXiv preprint arXiv:2510.10766*, 2025.

[7] V. U. Ihekoronye, S. O. Ajakwe, J. M. Lee, and D.-S. Kim, "DroneGuard: An Explainable and Efficient Machine Learning Framework for Intrusion Detection in Drone Networks," *IEEE Internet of Things Journal*, vol. 12, no. 7, pp. 7708–7725, Apr. 2025.

[8] Ş.BahtiyarandI.İşleyen, "GPS Spoofing Detection on Autonomous Vehicles with XGBoost," in *Proceedings of the 9th International Conference on Computer Science and Engineering (UBMK)*, IEEE, 2024.

[9] U.S. Department of Justice, Office of Justice Programs, National Institute of Justice. *Electronic Monitoring Reduces Recidivism*. NIJ In Short, NCJ 234460, September 2011.

[10] قنيش، مختار؛ بن عبو، جيالليي. *نظام المراقبة الإلكترونية باستعمال تقنية السوار الإلكتروني: دراسة حالة مؤسسة إعادة التربية والتأهيل معسكر*. دفاتر السياسة والقانون، المجلد 14، العدد 03، 2022، ص ص 114–121.

[11] Bostan, Ionel. *Electronic Surveillance in Court Proceedings and in the Execution of Criminal Penalties: Legislative and Logistical Steps Regarding Operationalising the Electronic Monitoring Information System (EMIS) in Romania*. Laws, 11(4), 54, 2022. https://doi.org/10.3390/laws11040054